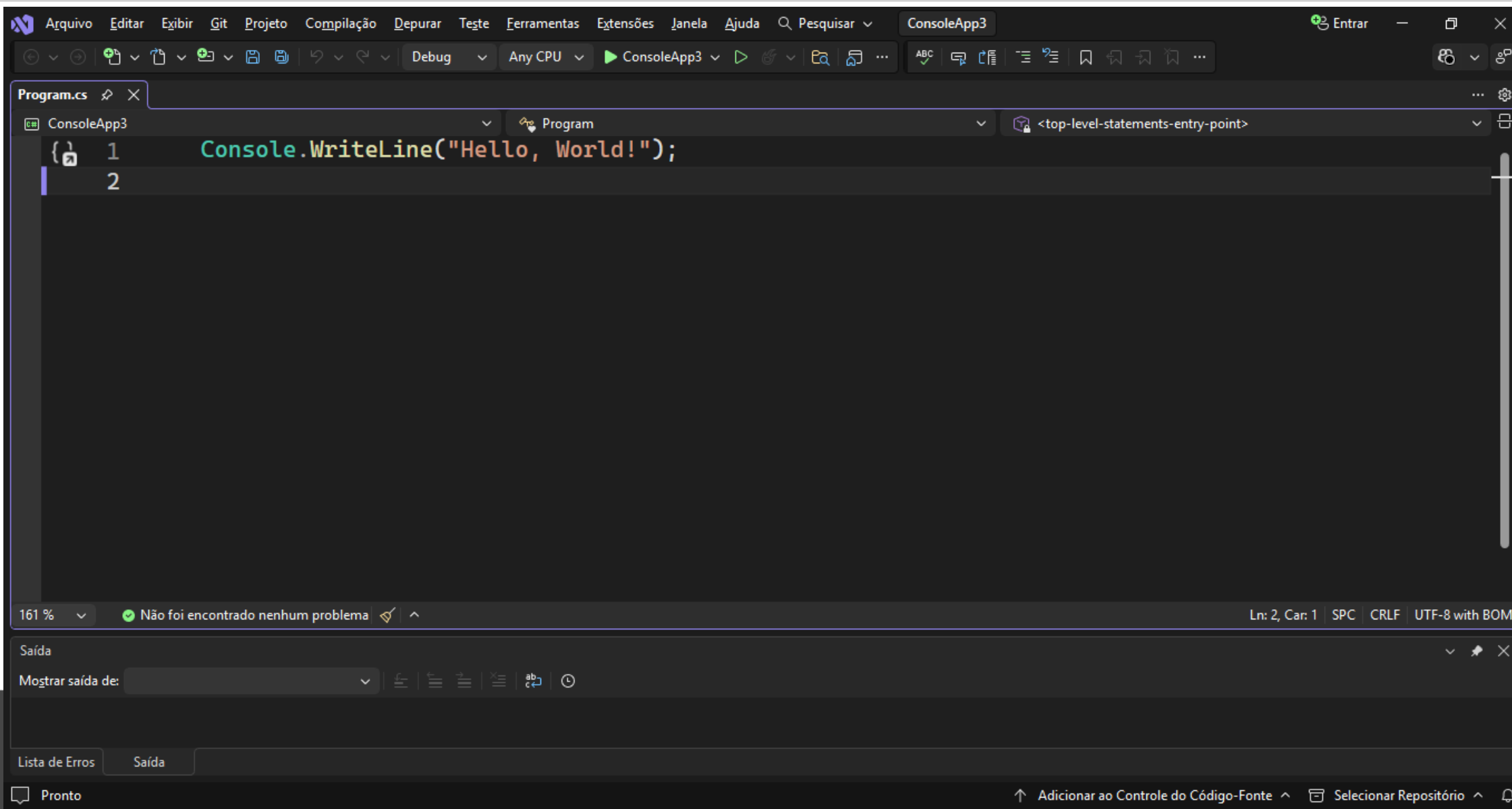


Técnicas de Programação

Explorando o ambiente do Visual Studio 2022 / 2026

- ☐ Apresentando o Visual Studio
- ☐ Formulários
- ☐ ToolBox, Solution Explorer, Properties
- ☐ Tipos de dados e formatação
- ☐ Operadores matemáticos, relacionais e lógicos
- ☐ Conversão de tipos
- ☐ Estruturas condicionais (if-else e switch-case)
- ☐ Estruturas de controle (while, do-while, for, foreach)
- ☐ Exercício de aplicação



Criar um novo projeto

Modelos de projeto recentes

-  Aplicativo do Console C#
-  .NET MAUI App C#

Pesquisar modelos (Alt+S)

Todos os idiomas ▼ Todas as plataformas ▼ Todos os tipos de pr... ▼



Aplicativo do Console
Um projeto para criar um aplicativo de linha de comando que pode ser executado no .NET no Windows, Linux e macOS

C# Linux macOS Windows Console



Aplicativo Web Blazor
Um modelo de projeto para criar um aplicativo Web Blazor que dá suporte à renderização do lado do servidor e à interatividade do cliente. Este modelo pode ser usado para aplicativos da Web com interfaces de usuário (UIs) dinâmicas avançadas.

C# Linux macOS Windows Blazor Nuvem Web



App de Inicialização Aspire (ASP.NET Core/Blazor)
Um modelo de projeto para criar um aplicativo Aspire com front-end web em Blazor e serviço de back-end em API Web, com cache Redis opcional.

C# API Aspire Blazor Nuvem Common MSTest NUnit Serviço Teste Web Web API xUnit



Aplicativo Web ASP.NET Core (Razor Pages)
Um modelo de projeto para criar um aplicativo ASP.NET Core com conteúdo de exemplo ASP.NET Core Razor Pages

C# Linux macOS Windows Nuvem Serviço Web



API Web do ASP.NET Core
Um modelo de projeto para criar uma API Web RESTful usando controladores ASP.NET Core ou APIs mínimas, com suporte opcional para OpenAPI e autenticação.

C# Linux macOS Windows API Nuvem Serviço Web Web API

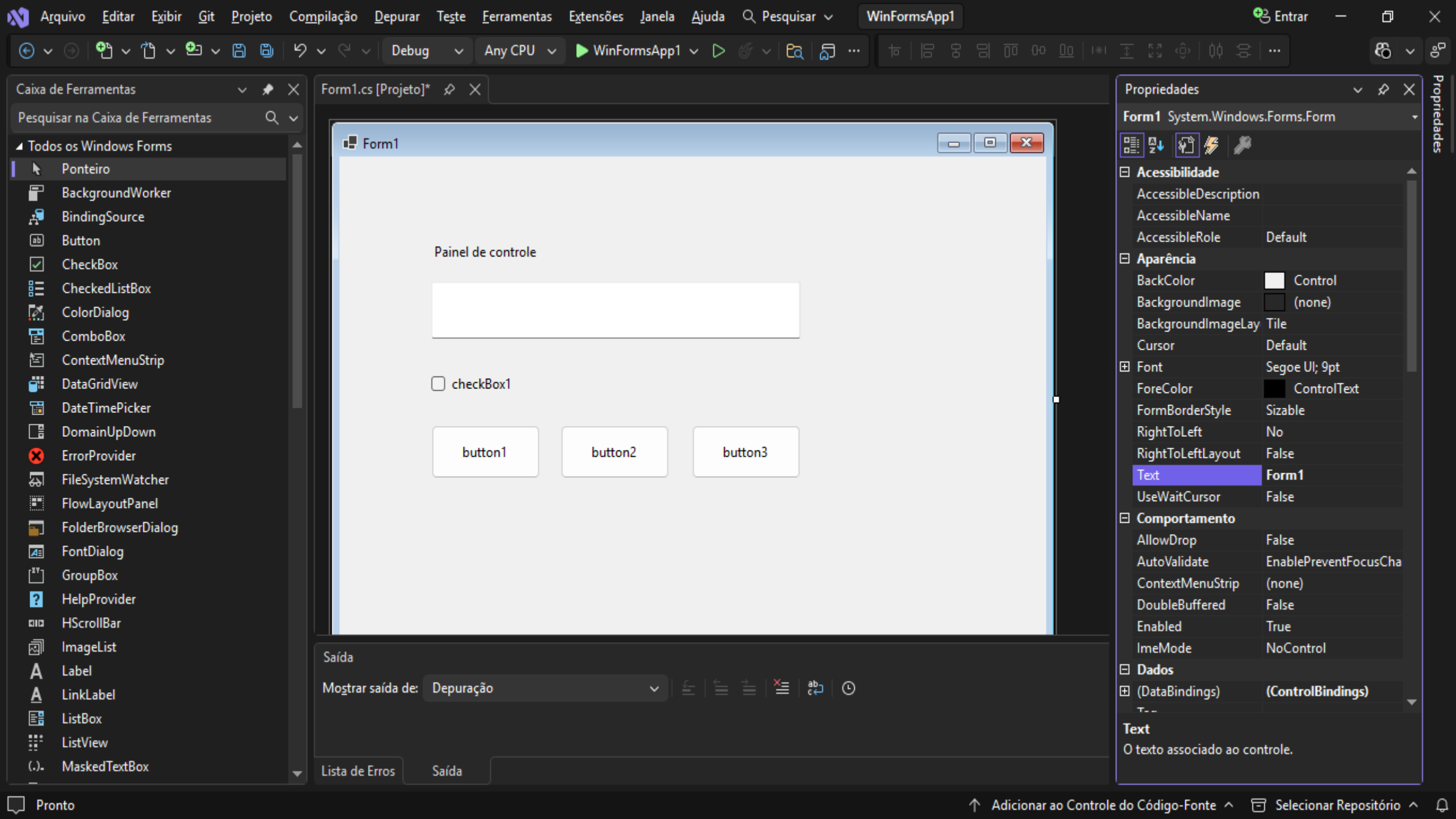


ASP.NET Core API Web (native AOT)
Um modelo de projeto para criação de uma API Web RESTful usando APIs mínimas ASP.NET Core publicadas como AOT nativas, com suporte opcional para OpenAPI.

C# Linux macOS Windows API Nuvem Serviço Web Web API

Atualização de Recursos março de 2026

[Voltar](#) [Próximo](#)





Microsoft
.NET

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace Media_Escolar
8 {
9     class Program
10     {
11         static void Main(string[] args)
12         {
13             double A, B, Media;
14
15             Console.WriteLine("Programa que calcula a média aritmética de dois valores!");
16             Console.Write("Digite um valor: ");
17             A = Double.Parse(Console.ReadLine());
18             Console.Write("Digite outro valor: ");
19             B = Double.Parse(Console.ReadLine());
20             Console.WriteLine();
21
22             Media = (A + B) / 2;
23             Console.WriteLine("A média dos dois valores é :" + Media);
24             Console.ReadKey();
25         }
26     }
27 }
28
```

Exemplo de
código fonte
em C#

Após a codificação, salvamos o projeto e utilizamos a tecla de atalho **F5** para compilar e executar o código.



Saída no console

C:\Program Files\dotnet\dotnet.exe

```
Programa que calcula a média aritmética de dois valores!
Digite um valor: 5
Digite outro valor: 8

A média dos dois valores é :6,5
```

❑ São nomes utilizados para identificar os elementos nos seus programas. Esses elementos podem ser:

❑ **namespace** (espaço de nomes), **classes**, **métodos**, **variáveis**, etc.

Podemos usar apenas letras, número e sublinhado para compor o nome dos identificadores.

❑ **Constantes**

const <tipo> <nomeDaConstante> = <valor>;

❑ Exemplos:

❑ const int numLinhas = 3;

❑ const int numColunas = 2;

❑ const double Pi = 3.14159265358979323846;

- ❑ Uma variável é definida como um lugar onde podemos armazenar dados temporariamente e usá-los posteriormente.
- ❑ Uma variável poderá armazenar um número inteiro, de ponto flutuante, um caractere, uma cadeia de caracteres (String) , um valor booleano, etc.
- ❑ O nome de uma variável pode conter letras, número e '_' (não pode haver sinais de pontuação e espaços em branco).
- ❑ Define o tipo de dados da variável
 - ✓ integer
 - ✓ byte, short, long
 - ✓ float, double, decimal
 - ✓ char, string
 - ✓ bool: true (verdadeiro), false (falso)

Declaração de variáveis

- int a, idade;
- char ch, caracter;
- float f, valor;
- double d;
- string s;

Toda variável deverá ser declarada explicitamente antes de ser usada

Tipo de dados	Intervalo
byte	0 ..255
sbyte	-128 ..127
short	-32,768 ..32,767
ushort	0 ..65,535
int	-2,147,483,648 ..2,147,483,647
uint	0 ..4,294,967,295
long	-9,223,372,036,854,775,808..9,223,372,036,854,775,807
ulong	0 ..18,446,744,073,709,551,615
float	-3.402823e38 ..3.402823e38
double	-1.79769313486232e308 ..1.79769313486232e308
decimal	-79228162514264337593543950335..79228162514264337593543950335
char	U+0000 .. U+ffff

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	get
goto	if	implicit	in	int
interface	internal	is	lock	long
namespace	new	null	object	operator
out	override	params	private	protected

public	readonly	ref	return	sbyte
sealed	set	short	sizeof	stackalloc
static	string	struct	switch	this
throw	true	try	typeof	uint
ulong	unchecked	unsafe	ushort	using

❑ Podemos atribuir o valor a uma variável usando o símbolo = (operador de atribuição). Exemplos:

int a = 1; // atribui o valor 1 a variável inteira a

float f = -8.76F; // observe o sufixo F

char c = "a"; // atribui o caracter a na variável c

double num = 2.55; // atribui o valor 2.55 na variável num

Propriedades de objetos

- value.text = 1; → **erro**
- value.text = "1"; → **ok**, porém o número um está sendo tratado como texto e não como número;
- value.text = **string.Parse**(1); • o número um é convertido de inteiro para uma string pelo uso do método parse da classe string.

As informações armazenadas em um tipo podem incluir o seguinte:

- O espaço de armazenamento que uma variável do tipo requer.
- Os valores mínimo e máximo que ele pode representar.
- Os membros (métodos, campos, eventos e etc.) que ele contém.
- O tipo base do qual ele herda.
- O local no qual a memória para as variáveis será alocada em tempo de execução.
- Os tipos de operações que são permitidos.

C#

 Copiar

```
int a = 5;  
int b = a + 2; //OK  
  
bool test = true;  
  
// Error. Operator '+' cannot be applied to operands of type 'int' and 'bool'.  
int c = a + test;
```

Quando declara uma variável ou constante em um programa, você deve especificar seu tipo ou usar a palavra-chave **var** para permitir que o compilador infira o tipo. O exemplo a seguir mostra algumas declarações de variáveis que usam tipos numéricos internos e tipos complexos definidos pelo usuário:

C#

 Copiar

```
// Declaration only:
float temperature;
string name;
MyClass myClass;

// Declaration with initializers (four examples):
char firstLetter = 'C';
var limit = 3;
int[] source = { 0, 1, 2, 3, 4, 5 };
var query = from item in source
            where item <= limit
            select item;
```

Tipo	Tamanho (em bytes)	Tipo .NET	Descrição
byte	1	Byte	Sem sinal (0-255)
char	2	Char	Caractere Unicode
bool	1	Boolean	Booleano (true=Verdadeiro ou false=Falso)
short	2	Int16	Com sinal (-32.768 a 32.767)
int	4	Int32	Com sinal (-2.147.483.648 e 2.147.483.647)
float	4	Single	Números de ponto flutuante. Armazena os valores de aproximadamente +/- $1,5 \times 10^{-45}$ até aproximadamente +/- $3,4 \times 10^{38}$ com sete número significativos. Usa o sufixo F
double	8	Double	Números de ponto flutuante de precisão dupla (com 15 a 16 números significativos)
decimal	16	Decimal	Valores monetários. Precisão fixa até 28 dígitos e a posição do ponto decimal. É normalmente usado em cálculos financeiros. Exibe sufixo "m" ou "M"
long	8	Int64	Números inteiros com sinal de - 9.223.372.036.854.775.808 a 9.223.372.036.854.775.807

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp2
{
    class Program
    {
        static void Main(string[] args)
        {
            byte a = 0;
            byte b = 255;

            char letra = 'T';

            bool teste = true;

            short numeral = 12546;

            float num1 = 12548.55F;

            double num2 = 12548.4579854587;

            decimal salario = 15487.25M;

            long numgrande = -945287458785878784;
```

```
Console.WriteLine("Os valores das variáveis são: " + "\n" +
    "O valor da variável a é: " + a + "\n" +
    "O valor da variável b é: " + b + "\n" +
    "O valor da variável letra é: " + letra + "\n" +
    "O Valor da variável teste é: " + teste + "\n" +
    "O valor da variável numeral é: " + numeral + "\n" +
    "O valor da variável num1 é: " + num1 + "\n" +
    "O valor da variável num2 é: " + num2 + "\n" +
    "O valor da variável salário é: " + salario + "\n" +
    "O valor da variável numgrande é: " + numgrande);

Console.ReadKey();
```

Selecionar C:\Users\Wagner\source\repos\ConsoleA...

```
Os valores das variáveis são:
O valor da variável a é: 0
O valor da variável b é: 255
O valor da variável letra é: T
O Valor da variável teste é: True
O valor da variável numeral é: 12546
O valor da variável num1 é: 12548,55
O valor da variável num2 é: 12548,4579854587
O valor da variável salário é: 15487,25
O valor da variável numgrande é: -945287458785878784
```


Especificador de Formato	Descrição	Exemplos	Saída
"C" ou "c"	Moeda	Console.Write("{0:C}", 2.5);	\$2.50
		Console.Write("{0:C}", -2.5);	(\$2,50)
D ou d	Decimal	Console.Write("{0:D5}", 25);	00025
"E" ou "e"	Científico	Console.Write("{0:E}", 250000);	2,500000E+005
F ou f	Ponto fixo	Console.Write("{0:F2}", 25);	25,00
		Console.Write("{0:F0}", 25);	25
"G" ou "g"	Geral	Console.Write("{0:G}", 2.5);	2.5
"N" ou "n"	Número	Console.Write("{0:N}", 2500000);	2.500.000,00
"X" ou "x"	Hexadecimal	Console.Write("{0:X}", 250);	FA
		Console.Write("{0:X}", 0xffff);	FFFF

++, --	Incremento, decremento
+, -, *, /, %	adição, subtração, multiplicação, divisão, módulo (resto)

Exemplo:

```
int a = 7;  
int b = 3;  
int x,y;  
x = a / b;  
y = a % b;
```

```
int x, y;  
x = 23;  
y = x++;           // operador pós-fixado  
Console.WriteLine(x + " " + y); // 23 24  
  
x = 23;  
y = ++x;           // operador pré-fixado  
Console.WriteLine(x + " " + y); // 24 e 24
```

Um operador booleano é um operador que faz um cálculo cujo resultado é true (verdadeiro) ou false (falso)

Operador	Ação
==, !=	Igual a, diferente de
> , >= , < , <=	Maior que, maior ou igual a, menor que, menor ou igual a
&&	“E” lógico (AND)
	“OU” lógico (OR)
!	“NÃO” lógico

Exemplo:

bool porcentagemValida = (porcentagem >= 0)

&& (porcentagem <= 100);

É possível converter um tipo de dados para outro tipo no C#. Podemos realizar conversão implícita de tipos (ou seja, o C# realiza a conversão de forma segura automaticamente) ou explícita (o programador utiliza funções pré-definidas - "cast"), ou ainda usar classes auxiliares.

Vamos ver um exemplo de conversão implícita:

```
int i = 10;  
Console.WriteLine("O número é " + i);
```

O conteúdo da variável *i*, do tipo inteira, é convertida implicitamente para o tipo **string** ao passá-la como **parâmetro** para o método **WriteLine** da classe Console.

```
int i = 90;  
float f = 22.100f;  
double d = 1450.33;  
bool a = true;
```

```
Console.WriteLine(i.ToDouble());  
Console.WriteLine(f.ToDecimal());  
Console.WriteLine(d.ToInt16());  
Console.WriteLine(a.());
```

Método	Conversão Realizada
ToBoolean	Converte um tipo em valor booleano, se possível
ToByte	Converte um tipo em Byte.
ToChar	Converte um tipo em caracter Unicode, se possível.
ToDecimal	Converte um ponto-flutuante ou um inteiro em decimal.
ToDouble	Converte um tipo em Double
ToInt16	Converte um tipo em inteiro de 16 bits.
ToInt32	Converte um tipo em inteiro de 32 bits.
ToInt64	Converte um tipo em inteiro de 64 bits.
ToString	Converte um tipo em String.

A classe System.Convert permite converter um tipo base em outro tipo de dados base usando seus métodos.

Exemplo:

```
int x = 15  
double y;  
y = Convert.ToDouble(x);  
Console.WriteLine("Valor convertido: {0} do tipo {1}", y, y.GetType());  
Console.WriteLine("Valor não convertido: {0} do tipo {1}", x, x.GetType());
```

```
String s = Console.ReadLine();
```

```
/* quando temos um número representado por uma string, não podemos fazer  
operações aritméticas com ele abaixo estão definidas as variáveis i, f e d */
```

```
int i;
```

```
i = int.Parse(s); //converte uma variável do tipo String em um inteiro
```

```
float f;
```

```
f = float.Parse(s); //converte uma variável do tipo String em um float
```

```
double d;
```

```
d = double.Parse(s); // converte uma variável do tipo String em um double
```

O if avalia uma expressão lógica booleana e qualquer outro tipo será acusado como erro pelo compilador. Se o resultado for verdadeiro, o bloco de código dentro do if será executado; caso contrário, o controle é passado para a próxima declaração após o if. A declaração if tem três formas básicas:

1. IF Simples

```
if (expressão)
{
    Declaração
}
```

2. IF - ELSE

```
if (expressão)
{
    Declaração
}
else
{
    Declaração
}
```

3. IF Aninhado

```
if (expressão)
{
    Declaração
}
else if (expressão)
{
    Declaração
}
```


// Exemplo1

```
int a = 0;
int b = 1;
if ( a < b )
{
    Console.WriteLine("B é maior");
}
else
{
    Console.WriteLine("A é maior");
}
```

//Exemplo2

```
int a = 0;
int b = 1;
if ( a < b )
{
    Console.WriteLine("B é maior");
}
else if ( a > b )
{
    Console.WriteLine("A é maior");
}
else
/* e finalmente a condição de igualdade deveria
ser satisfeita */
{
    Console.WriteLine("A é igual a B");
}
```

A declaração switch avalia uma expressão cujo resultado pode ser dos tipos sbyte, byte, short, ushort, int, uint, long, ulong, char, string ou enum, e este por sua vez é comparado com cada uma das seções case que constituem o switch. Vejamos a sua sintaxe:

```
switch(expressão)
{
    case constante1:
        declaração 1;
        break;
    case constante2:
        declaração 2;
        break;
    ...
    default:
        declarações;
        break;
}
```

```
char letra = 'b';
string resposta = " ";
switch( letra ) {
    case 'a':
        resposta = "Resposta errada!";
        break;
    case 'b':
        resposta = "Resposta correta!";
        break;
    default:
        resposta = "Resposta inválida!";
}
meuLabel.Text = "resposta";
```

```
int opcao;  
Console.WriteLine("Digite a opção desejada: ");  
opcao = int.Parse(Console.ReadLine());  
switch (opcao)  
{  
    case 0:  
        Console.WriteLine("A é igual a B");  
        break;  
    case 1:  
        Console.WriteLine("A é menor do que B");  
        break;  
    case 2:  
        Console.WriteLine("A é maior do que B");  
        break;  
}  
Console.ReadKey();
```

1. Em C#, é obrigatório que cada seção case tenha uma declaração **break**.
2. A seção **default**, que é avaliada caso nenhuma das seções case for verdadeira, não é obrigatória.
3. Não pode existir mais de uma seção case com a **mesma constante**.

O laço **while** é usado quando não sabemos o número de vezes que devemos executar um bloco de código, mas apenas a condição que deve ser satisfeita para executar o bloco dentro do **while**. Essa condição é uma expressão booleana que deverá ser verdadeira para garantir pelo menos a primeira ou a próxima iteração.

Exemplo:

```
int i = 95;           // declara a variável i e atribui o valor 95
while ( i < 100) // testa a condição usando o laço While (enquanto)
{
    Console.WriteLine(" {0}", i); // Escreve no console o valor de i
    i++; // incrementa a variável i, aumentando de 1 em 1
}
```

 C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

```
95
96
97
98
99
```

```
int contador = 0;
int valor = 0;
while ( contador < 10 ) {
    valor += 10;
    contador ++;
}
meuLabel.Text = valor.ToString( );
int contador = 0;
int valor = 0;
while ( contador < 10 ) {
    valor += 10;
    contador ++;
}
meuLabel.Text = valor.ToString( );
```

```
int n = 0;
while (n < 5)
{
    Console.WriteLine(n);
    n++;
}
```



C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

```
0
1
2
3
4
```

O laço do ... while (faça – enquanto)

Este tipo de laço é usado quando queremos que um bloco de código seja executado pelo menos uma vez. A condição a ser satisfeita se encontra no fim do bloco de código e não no começo, como no caso dos laços for e while.

Exemplo:

```
int i = 95;
```

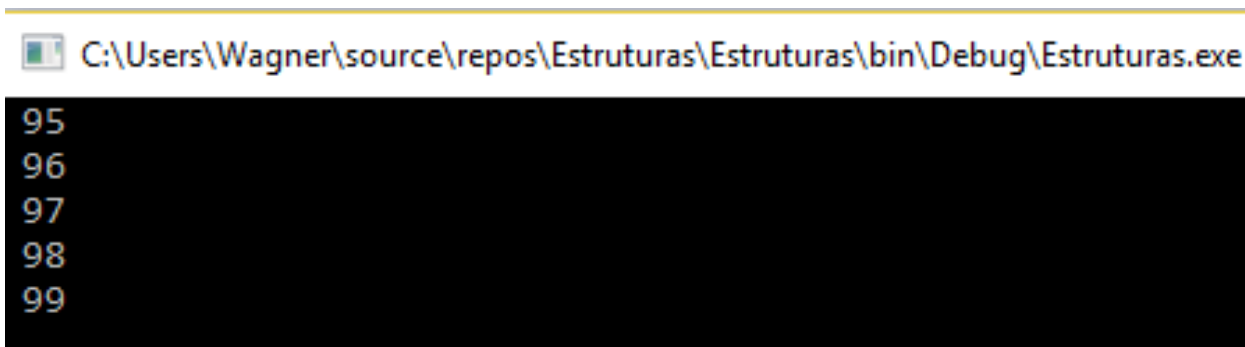
```
do
```

```
{
```

```
    Console.WriteLine("{0}", i);
```

```
    i++;
```

```
} while ( i < 100)
```



```
C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe  
95  
96  
97  
98  
99
```

No exemplo acima, se o valor da variável i fosse igual a 100 antes da execução de do/while, o laço seria executado pelo menos uma vez, o que não aconteceria se usássemos while.

A palavra reservada continue permite que o fluxo de execução da iteração corrente seja abandonado, mas não o laço, e a iteração seguinte dê início no topo do laço, uma vez que a condição do for seja satisfeita.

Exemplo:

```
for(int i=0; i<=5; i++) {  
    Console.WriteLine("Iteração número {0}", i);  
    if (i == 3)  
        break;
```

```
}
```

```
for(int i=0; i<=5; i++) {  
    Console.WriteLine("Iteração número {0}", i);  
    if (i == 3)  
        continue;
```

```
// a declaração a seguir não será executada quando i==3
```

```
Console.WriteLine ("Iteração número {0}", i +2 );
```

```
}
```

```
Iteração número 0  
Iteração número 1  
Iteração número 2  
Iteração número 3
```

```
Laço for com continue!  
Iteração número 0  
Iteração número 2  
Iteração número 1  
Iteração número 3  
Iteração número 2  
Iteração número 4  
Iteração número 3  
Iteração número 4  
Iteração número 6  
Iteração número 5  
Iteração número 7
```


Laços aninhados são laços dentro de laços que podem ser usados, por exemplo, (como já o fizemos anteriormente) para varrer **arrays** multidimensionais.

Exemplo:

```
int i, j;  
for (int i=0; i < 10; i++)  
{  
    for (int j=0; j < 10; j++)  
    {  
        Console.WriteLine("{0}\t", i, j) ;  
    }  
}
```

Se uma declaração **break** estiver no laço interno, este será abandonado e o controle será passado para o laço externo; mas se estiver no laço externo, os dois laços serão abandonados e o controle passará para a próxima declaração após o laço.

0	0	0	0	0	0	0	0	0	0	1	1	1	1	1
1	1	1	1	1	2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3	4	4	4	4	4
4	4	4	4	4	5	5	5	5	5	5	5	5	5	5
6	6	6	6	6	6	6	6	6	6	7	7	7	7	7
7	7	7	7	7	8	8	8	8	8	8	8	8	8	8
9	9	9	9	9	9	9	9	9	9					

Este tipo de laço é usado para varrer **arrays** ou **coleções**.

```
class Class1
{
    static void Main(string[] args)
    {
        string[ ] nomes = { "Ana", "Maria", "João", "José" };
        foreach (string pessoa in nomes)
        {
            Console.WriteLine(" {0} ", pessoa);
        }
        Console.ReadKey();
    }
}
```

C:\Users\Wagner\source\repos\Estruturas\Estruturas\bin\Debug\Estruturas.exe

Ana
Maria
João
José

Estruturas de repetição

while

do...while

for

foreach

break

continue

Comando While

1

2

3

4

5

6

7

8

9

10

11

12

13

Estruturas de repetição

while

do...while

for

foreach

break

continue

Comando Do/While

20

19

18

17

16

15

14

13

12

11

10

9

8

Button
Name: btnWhile
Text: While
Size: 180x40

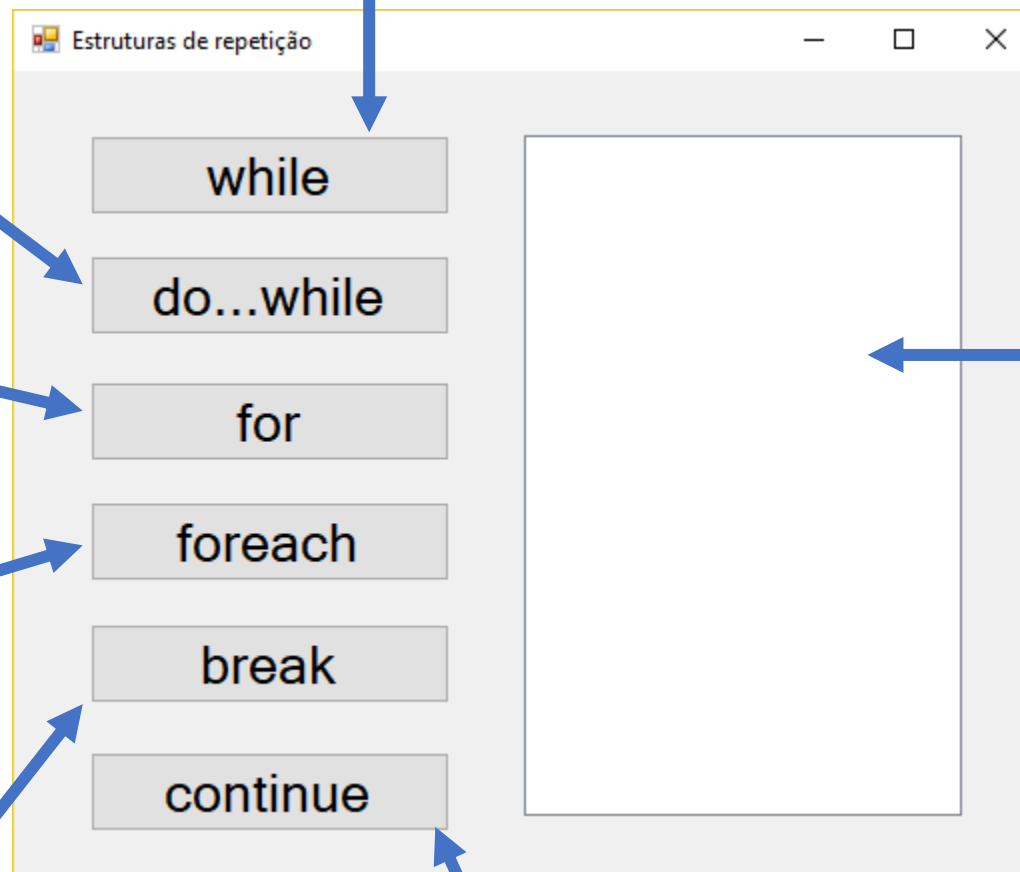
Button
Name: btnDoWhile
Text: do...while
Size: 180x40

Button
Name: btnFor
Text: for
Size: 180x40

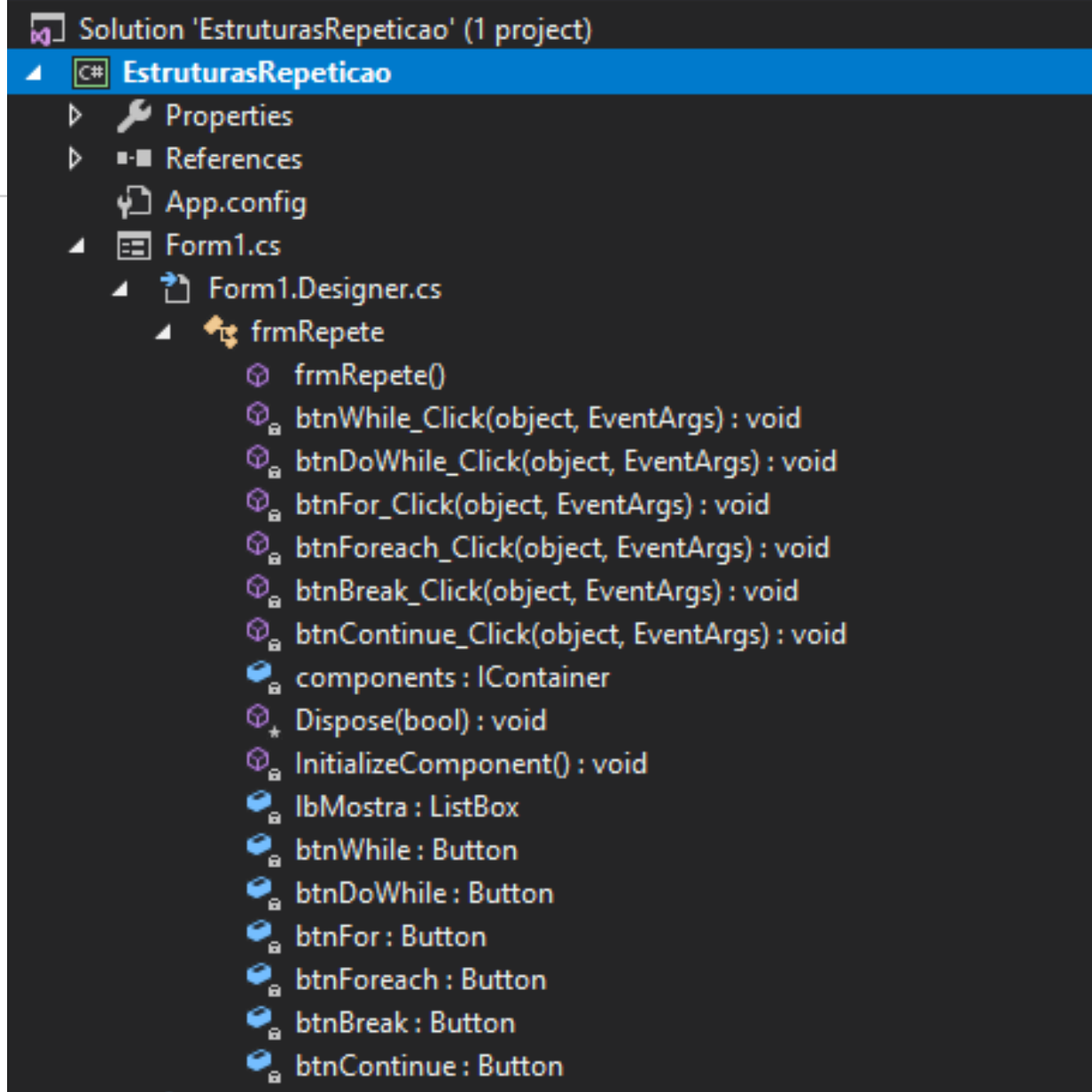
Button
Name: btnForEach
Text: foreach
Size: 180x40

Button
Name: btnBreak
Text: break
Size: 180x40

Button
Name: btnContinue
Text: continue
Size: 180x40



ListBox
Name: lbMostra
Text: ListBox
Size: 219x340



Solution Explorer

while

```
#region While
1reference
private void btnWhile_Click(object sender, EventArgs e)
{
    int numero = 1;
    lbMostra.Items.Clear();
    lbMostra.Items.Add("Comando While");
    while (numero <= 20)
    {
        lbMostra.Items.Add(numero.ToString());
        numero += 1;
    }
}
#endregion

1reference
private void btnDoWhile_Click(object sender, EventArgs e)
{
    int numero = 20;
    lbMostra.Items.Clear();
    lbMostra.Items.Add("Comando Do/While");
    do
    {
        lbMostra.Items.Add(numero.ToString());
        numero -= 1;
    }
    while (numero >= 1);
}
```

do...while

```
1 reference
private void btnFor_Click(object sender, EventArgs e)
{
    lbMostra.Items.Clear();
    lbMostra.Items.Add("Comando For");
    for (int i = 1; i <= 20; i++)
    {
        lbMostra.Items.Add(i.ToString());
    }
}
```

```
1 reference
private void btnForeach_Click(object sender, EventArgs e)
{
    lbMostra.Items.Clear();
    lbMostra.Items.Add("Comando Foreach");
    ArrayList lista = new ArrayList();
    for (int i = 1; i <= 20; i++)
    {
        lista.Add("Número_" + i.ToString());
    }
    foreach (var item in lista)
    {
        lbMostra.Items.Add(item);
    }
}
```

for

foreach

```
private void btnBreak_Click(object sender, EventArgs e)
{
    lbMostra.Items.Clear();
    lbMostra.Items.Add("Comando Break");
    for (int i = 1; i <= 20; i++)
    {
        lbMostra.Items.Add(i.ToString());
        if (i == 10)
        {
            break;
            MessageBox.Show("", "Comando Break, nº" + i.ToString());
        }
    }
}
```

1 reference

```
private void btnContinue_Click(object sender, EventArgs e)
{
    lbMostra.Items.Clear();
    lbMostra.Items.Add("Comando Continue");

    for (int i = 1; i <= 20; i++)
    {
        if ((i == 10) || (i == 11) || (i == 12))
            continue;

        lbMostra.Items.Add(i.ToString());
    }
}
```

break

continue



Perguntas & Respostas